

Developing Complex Projects Using XP with Extensions



Using methodological extensions to adapt XP for large-scale or long-term projects offers a high degree of security and reliability without limiting the advantages of agile software development.

*Martin
Lippert*

*Petra
Becker-
Pechau*

*Holger
Breitling*

*Jörn
Koch*

*Andreas
Kornstädt*

*Stefan
Roock*

*Axel
Schmolitzky*

*Henning
Wolf*

*Heinz
Züllighoven*
IT Workplace
Solutions

Extrême Programming is a disciplined approach to software development that emphasizes customer satisfaction and teamwork,¹ but IT managers often view it as a slightly chaotic methodology. Many even regard XP as dangerous and unpredictable because to them it appears to neglect planning and controlling in large-scale or long-term projects.²

When asked about his previous impressions of XP, a participant in one of our IT workgroups described it as “structured anarchy.” This comment reflects the broadly held view that XP is a loosely regulated process that gives developers in particular a great deal of freedom. Many developers find this feature attractive, but to project leaders and managers, “loosely regulated” is often synonymous with “unplanned and uncontrollable.”

However, our experience contradicts this view. When suitably adapted for use in projects with complex domains or limited resources, we have found that XP offers a high degree of security and reliability without limiting the advantages of agile software development.^{3,4}

At IT Workplace Solutions (www.it-wps.com), we have followed the recommended practice of adapting XP to each specific project⁵ and have developed methodological extensions to XP for use in a number of areas in which questions and problems frequently occur. We apply these extensions in a cyclic and iterative approach that emphasizes constant feedback, project preparation, and the creation of new roles for the participants in a software development project.

In our view, this approach provides a way to deal with both the technical complexity of an application domain and the complexity inherent in a development process involving many groups. In addition to extending the application of XP to manage projects in which fuzzy requirements may hamper establishing realistic release schedules, it also helps to prepare customers for the effort involved in testing by addressing the critical issue of resource management.

Table 1 summarizes some projects in which we have used our approach. Each project represents a unique combination of characteristics including the front-end system (Web, desktop, batch processing, or mobile clients), location (in-house or external), number of developers and coaches, participant qualifications, duration, and special features.

PLANNING FOR MULTIPLE CUSTOMERS

A major problem with conventional development processes is that they leave a wide gap between customers and developers. To prevent misunderstandings about a project's objectives and outcome as early as possible, XP and other agile processes rely on the customer's close involvement throughout the development process.¹

In large projects, however, constructing the software requires meeting the needs of numerous potential user groups. Having only one XP team involved in meeting these diverse needs can strain the project's limited resources; on the other hand, if the project involves several teams, the challenge is to reconcile the various stakeholders' interests to develop a uniform vision for building the software.

Table 1. Project characteristics.

Project	Front end(s)	Project location	Number of developers/coaches	Participant qualifications	Duration (consumed/rest)	Special features
A	Web, desktop, batch processing	In-house	15/2	Highly heterogeneous; participants occasionally conversant with technologies	24/12 months (200/200 person-months)	Different interest groups; multichanneling application
B	Desktop, batch processing	External	6/0	Homogeneous, but initially little domain knowledge	6 months; completed	High demands with regard to documentation for certification by auditor(s)
C	Desktop, mobile clients	External	10/0	Homogeneous	9/27 months (40/100 person-months)	Project divided into two phases: demonstrate feasibility/utility and prototype development
D	Desktop	External	6/0	Homogeneous	6 months; completed	Product development
E	Batch processing	In-house	3/1	Heterogeneous	15 months; completed	Developers' first experience with OO; embedded in host environment

Product managers

In Projects A and C, several business divisions approached the XP teams with unfiltered, sometimes conflicting requirements for a specific construction object.

One solution to this problem is to appoint a *product manager*, who determines the requirements of the various interest groups. The product manager then plays the role of the XP customer in a *planning game*, which is conducted to decide the scope and timing of releases based on estimates the programmers provide. Following the planning game, the various interest groups receive a summarized feature list.

Product managers can improve many projects, but making compromises when conflicts occur requires special skills. Evaluating a product manager's performance is difficult, but the users' reactions at system presentation usually are a good indicator. Although occasional complaints about missing features are acceptable, continual dissatisfaction with feature implementations might indicate the need to replace the product manager.

To avoid communication problems, if developers have purely domain-related questions rather than questions about task prioritization, they should circumvent the product manager and query the users directly.

Phased parallel construction

An additional problem in Project C was the limited size of the XP team, which could not build software simultaneously for all three of the business divisions involved in the project. In the planning game, it was not clear which division should receive priority, what effort this required, or how to divide the resources among the three groups.

Initially, as Figure 1a shows, the team suggested that it should allocate three weeks to each of the

three business divisions in turn. However, the customers immediately rejected this suggestion because no work would be done on their system for a period of six weeks.

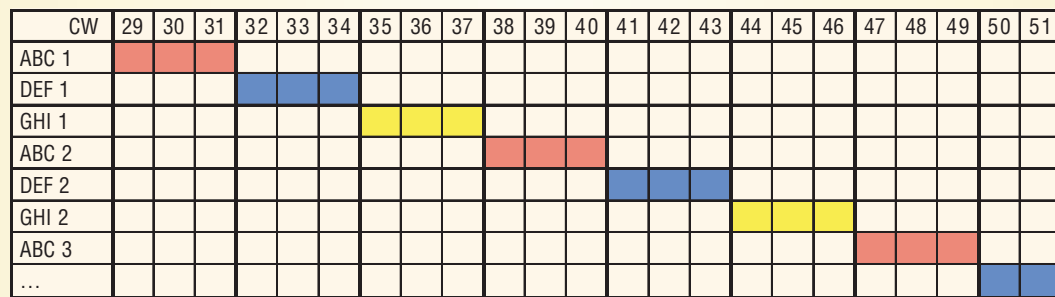
In this case, the solution was *phased parallel construction*, with the development team working on two business divisions in parallel. As Figure 1b shows, for each division, the work was conducted over a total six-week period in a staggered three-week rhythm. Rather than increasing the construction work for each division, this approach merely stretched it out, reducing the break for each division to an acceptable three-week period. The increased processing period facilitated closer feedback with the customers from each division and avoided the misunderstandings that can occur with a lengthy break.

To use phased parallel construction effectively, developers must gain insight into each interest group's work situation. Misunderstandings can occur because similar terms have different meanings for different interest groups.

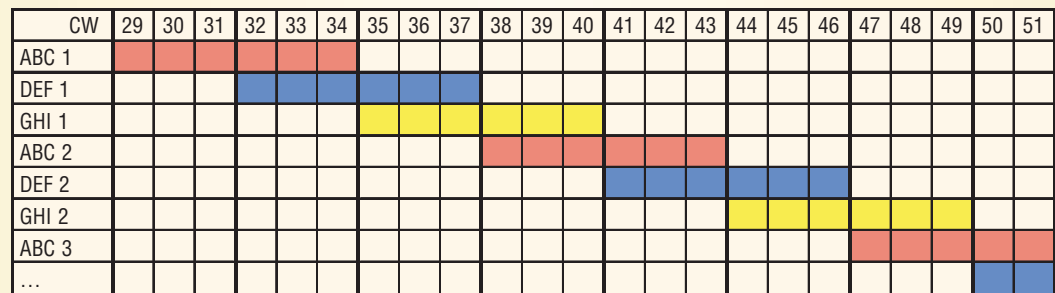
Distribution over multiple teams

Projects A and C posed a similar problem: Several business divisions presented their requirements, and none wanted to wait long for their implementation. Some groups repeatedly complained that their requirements received too little attention, or that it came too late.

For Project A, a separate XP team was established for each business division to determine the amount of resources available. A product manager was assigned to each team to consolidate the conflicting requirements of the various interest groups. The product managers then represented their group by participating in separate planning games conducted for each team.



(a)



(b)

Figure 1. Building software for three business divisions (ABC, DEF, and GHI). (a) Allocating three weeks to each division in turn would result in no work being done on a given system for six weeks at a time. (b) Phased parallel construction stretches out each division's workload and enables customers to provide closer feedback. CW = calendar week.

Distribution over several XP teams—each with, as we propose, at least four developers—offers a viable option when phased parallel construction reaches its limit. Nevertheless, having several small teams working together in a project invariably causes communication overhead.

CUSTOMER TESTING

Using customer feedback to swiftly check decisions and developments is a key principle of agile development processes such as XP.

Frequent releases are particularly important because customers can directly evaluate the project's results. Unlike system presentations, using early system versions helps to address questions relating to features such as stability, load behavior, and performance. Customer reluctance to agree to develop releases in small increments reduces the effectiveness of this important risk management instrument.

Problems can occur if the customer does not follow through on promises to extensively test the prototypes and subsequent versions. When faults in the developed software do not become evident until shortly before the planned introduction of the system, fixes must be made under significant time pressure.

Preparing the customer for the testing effort

Although most customers do not dispute the benefits of early releases, they often fail to carry out testing because it appears to offer little direct benefit in return for the time and resources it requires. However, early and frequent feedback is an essential XP concept.

Our approach emphasizes the importance of holding a frank conversation at the beginning of

the project that explains what is expected of the customer, particularly with respect to regular involvement in testing. This conversation should make clear how the customer can integrate this effort into a normal work routine and that the testing will yield tangible benefits. Responding to objections about testing provides an opportunity to initiate an early search for solutions that foster the XP goal of encouraging the customer's close involvement in the development process.

More up-front risk management

More intensive risk management is required if the customer does not—or cannot—use the developed releases. It is difficult or impossible to use system demonstrations to detect risks related to load behavior, robustness, and the support of cooperative work. Detecting these risks as early as possible is essential, but developers can only find them if they look for them. The development team should follow the XP principle of building spike solutions as simply as possible, separate from system development, while producing as little Yagni (“you aren’t going to need it”) code as possible.

However, spike solutions cannot answer questions about the system's practical worth. Answering such questions requires putting the release to use.

Providing documentation and feedback on test results

Even if the customer does carry out testing, the results from manual acceptance tests can be interpreted in different ways. At this point in the project, developers must assume a subordinate role because the customer determines what is good and correct. Developers must be able to assess the quality of the

Combining customer-on-demand with the author-critic cycle allows sufficiently close and swift communication to avoid delays in the project.

customer testing, but doing so is a difficult undertaking, especially when the feedback is positive.

One solution is to both document the test setup and results and to evaluate them in a feedback cycle. Feedback can point to remaining weaknesses in the tests, providing a basis for more meaningful tests. While endorsed test results offer more protection against unwarranted additional requirements, the feedback cycle is beneficial because high-quality acceptance tests avoid the need for such requirements.

COMPLEX OR IMPRECISE APPLICATION DOMAIN

We generally assemble development teams as needed. Because team members work for a limited period on a project within a specific domain, they tend not to be specialists in any particular domain. This can cause problems when developing software for a complex domain because developers initially may not understand how the domain works or what the system is supposed to accomplish.

To design and implement the new system, the development team must work with the customer in a joint learning process to understand the system requirements and to translate them into the software. The customer's availability is a critical issue when the project requires specialist knowledge about the domain.

Author-critic cycles

XP's iterative and incremental approach focuses on the source code. However, in complex long-term projects, developing other types of documentation in short cycles and having the customer assess them is beneficial.

Project B, for example, required the specification and documentation of an assets accounting system to provide a basis for certification by an auditor. Instead of selecting a typical phase-oriented approach, the developers gradually improved results in short cycles and regularly requested customer feedback.

In informal interviews with the customer, the developers first clarified essential domain-related issues and documented the results in their own words but taking care to use the terms and concepts of the application domain. The customer reviewed these results to provide feedback and point out any errors. The developers incorporated this feedback into a revised version of the documents and again presented them to the customer to

review. This *author-critic cycle* continued through a number of iterations.⁶

This document-based process was particularly important in the early stages of Projects B and E. By closely communicating with the customer and recapitulating the results, the developers gained more detailed specialist knowledge with every cycle. Both projects produced technical glossaries; in addition, Project E generated a detailed description of the implemented calculation procedures, and Project B generated certifiable system documentation.

The author-critic cycle requires additional effort and skills. Developers must be familiar with writing and reviewing documentation in the language of the application domain. In addition, both customers and developers must expend considerable time and effort on nonprogramming tasks, without overdoing it. When executed correctly, the author-critic cycle creates a systematic information base of high-quality documents that are readily available to project members.

Customer-on-demand

For projects in which the customer is not available full-time onsite, we rely on the *customer-on-demand* concept to facilitate communications. Using this approach, developers usually can contact customers by telephone within a few minutes and, if necessary, schedule a physical meeting on the same or the following day. Combining customer-on-demand with the author-critic cycle allows sufficiently close and swift communication to avoid delays in the project.

LONG-TERM PLANNING

XP planning instruments focus on individual iterations and releases. During XP's planning game, team members identify and assess features as *stories*, a summary of information about individual use cases that will fit on an index card. Cost and speed of implementation for each story is estimated based on experience from either previous stories of the same project or previous projects in which the problems and team structures were similar.

Exploratory phase with subsequent rough release schedule

The more a project differs from comparable previous projects in terms of its goals, technologies used, and team composition, the more difficult it is to provide reliable estimates over a long period. Project A, for example, involved replacing a large legacy system with a new system that could do "all that the

old system could, but better,” but nobody on the team could provide reliable information about the old system’s functionality. Team members had patchy application-related knowledge, and most were unfamiliar with the technologies. In this case, the collective lack of experience and project’s size made the exclusive use of story cards impractical.

Large projects requiring some guarantee of feasibility within budgetary constraints call for an initial one- to three-month *exploratory phase*, during which developers conduct interviews and construct prototypes⁶ to survey the field as much as possible. On the application side, this phase provides an opportunity to assess expected functionality; on the technical side, the developers obtain an overview of the existing and proposed technology.

The development team can use the information gathered in the exploratory phase to better assess the project’s scope and draw up a rough feature list that they partition into *task packages*. The team also gets a better sense of what it can achieve, at least in the near term.

The next step is to draw up a *rough release schedule* for the entire project, including estimates of the effort involved for each individual release. The accuracy in determining the first release serves as an indication of how realistic the initial planning was. A high degree of accuracy reinforces the conviction, both within the team and on the part of management, that the stated goal is attainable. On the other hand, substantial deviations may require adjusting the entire schedule if eliminating the interference factors through training or other measures is impossible. Gradually establishing better assessment planning increases reliability with each release.

Some XP practices are not applicable during the exploratory phase, but others—such as stand-up meetings, refactoring, collective ownership, and pair programming—are definitely valuable. The entire phase is essentially one high-level planning game for the upcoming project. However, we do not consider an exploratory phase to be a breach of XP’s *no upfront design* principle because no details are fixed in advance.

Management generally prefers to add some extra days of exploration rather than to start a project when the development team does not even know whether its task is feasible. An often-heard piece of advice⁷ regarding functionality assessment also applies: Speak to the right people. Management may think it knows best, but the users’ acceptance ultimately decides the project’s fate. It is important to select experienced people who are familiar with the system. In projects with multiple customers, the

exploratory phase is also a good time to look for a product manager.

DEPENDENCIES AMONG PROJECT TASKS

Two key principles of XP are to focus on the program code and to neglect dependencies among programming tasks.¹ This is why story and task cards are not used except for programming. However, software projects involve many other nonprogramming tasks that are often interdependent and can strongly influence the development process. Such tasks—including presentations on the project’s progress, training team members in the use of new technologies, hardware and software procurement, and planning production takeover—must be integrated into development process planning.

Project A, for example, focused so intently on the planning game that the customer’s failure to do his part to introduce the new system did not become apparent until relatively late. The program code was on schedule, but the customer had not yet purchased the computer on which the project’s Web server was to be installed. It took several months—an unexpectedly long time—to procure a suitable computer, unnecessarily holding up overall project progress by delaying acceptance testing by the application department.

Baselines with responsibilities

To address this problem, the team can use *baselines* to model all nondevelopment project tasks. A baseline consists of a task description, a list of possible dependencies on other baselines, the name of a person responsible for meeting each baseline, and, in some cases, an assessment of effort.⁶ The person responsible for a baseline does not necessarily perform the task; that person’s job is simply to keep an eye on task performance and keep it moving along.

A *baseline manager* monitors overall progress toward all baselines, using a prominently displayed table to identify new tasks and dependencies in cooperation with the development team and customer. After each planning game, the baseline manager determines whether to include tasks not captured on story cards in the table, ticks off baselines reached, and reminds those responsible of their duties.

Adding baselines complicates the process because using both stories and baselines influences the planning game and task assignment. Participants in the planning game prioritize task assignments and balance the resources available for stories with those needed to monitor and complete the baselines.

The development team can use the information gathered in the exploratory phase to better assess the project’s scope and draw up a rough feature list that it partitions into task packages.

Business story lists constitute a good basis for managing and monitoring software projects because management and the customer can agree on exactly what has been accomplished.

Developers must choose between selecting the next story card or working on a baseline.

One possible solution is to assign one or two persons to work exclusively on baselines. Because baseline tasks address infrastructure or nondevelopment tasks that may not be obvious to the customer, the project cannot rely on the customer to define necessary baselines in the planning game. The team should also suggest baseline tasks. Whether the customer or the project manager determines tasks and prioritizes baselines depends on the project.

REQUIREMENTS DEFINITION

In the XP approach, customers write story cards that capture the system requirements during the development process. However, unlike traditional requirements specifications, these story cards do not represent exact definitions.⁵ Capturing requirements of proper granularity at the outset is sometimes necessary, especially in conventional fixed-price projects, to use as a basis for the contract or for long-term planning. These requirements must be detailed enough to allow a rough effort assessment but not so precise as to stifle developers' creativity.

Business stories

During the project's exploratory phase, the developers collaborate with user representatives from each business division or subproject to draw up the aims as well as special "requirements documents." The aims roughly define what is expected of the project as a whole—which business processes it will support and what aspects need improvement. In contrast, the requirements documents are lengthy stories indicating the future system's anticipated functionality. These *business stories* tend to be less detailed than normal stories, and they are more closely geared to business processes than to user tasks.

For example, in Project C, the three business divisions formulated their different system requirements as typical business processes—lists of previously used work materials and generic activities to perform on them such as create, read, update, or delete. However, it was not clear how the team would use these general descriptions to arrive at concrete system requirements.

Each of the three subprojects produced 35 business stories in a project runtime of nine months, requiring approximately 600 person-days of developer capacity. The story list was divided into two sections: necessary and optional stories. While only the necessary stories are relevant to acceptance in

contractual terms, capturing the optional stories helps team members look beyond the current project's confines.

Instead of using a very general requirements specification, the customer and provider agreed to use these business stories on a contractual basis. By mutual arrangement, they also agreed that the customer could alter the business stories, even during the course of the project, as long as such changes did not increase overall development effort. Thus, for every new requirement, an old one had to be abandoned or simplified. This arrangement makes it possible for the customer to adjust aims while a project is in progress, but it does not put the developers at a disadvantage because the requirements do not increase, they merely change.

Basically, such business story lists constitute an *optional scope contract*,⁸ the terms of which can be altered during a project. However, to prevent confusion, we do not recommend using this term in the initial negotiations with the customer. This does not imply reducing communication; on the contrary, business stories require quite intensive negotiations, but it is more effective to introduce this concept after the customer has learned to trust the provider.

Business story lists constitute a good basis for managing and monitoring software projects because for each business story, management and the customer can agree on exactly what has been accomplished. They also provide a sound basis for iteration and release scheduling.

Because business stories are at a higher level of abstraction than story cards, team members must maintain contact with lower-level project planning to ensure flexibility.

We don't invariably use XP on every project. Instead, we adapt the development method to each situation and incorporate new elements in our process as needed. The XP extensions we have created focus on the planning and controlling aspects of development, demonstrating that, contrary to the opinion of many IT managers, a suitably adapted agile development process is ideal for long-term projects and the development of large systems. ■

Acknowledgment

We thank all it-wps coworkers and project partners for the numerous discussions on the topics addressed here and for their help in gathering experience.

References

1. K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2000.
2. A. Cockburn, *Agile Software Development*, Addison-Wesley, 2002.
3. M. Lippert et al., "XP in Complex Project Settings: Some Extensions," *Extreme Programming Perspectives*, M. Marchesi et al., eds., Addison-Wesley, 2001, pp. 579-589.
4. M. Lippert, S. Roock, and H. Wolf, *Extreme Programming in Action: Practical Experiences from Real-World Projects*, John Wiley & Sons, 2002.
5. K. Beck and M. Fowler, *Planning Extreme Programming*, Addison-Wesley, 2001.
6. H. Züllighoven, *The Object-Oriented Construction Handbook*, Morgan Kaufmann, to be published 2003.
7. B. Meyer, *Object-Oriented Software Construction*, 2nd ed., Prentice Hall, 1997.
8. K. Beck and D. Cleal, "Optional Scope Contracts"; www.xprogramming.com/ftp/Optional+scope+contracts.pdf.

Martin Lippert is a software architect and coach at IT Workplace Solutions (it-wps), an IT consulting and development company in Hamburg, Germany, and a research assistant and PhD candidate in the Software Engineering Group (SWT) of the University of Hamburg's Department of Computer Science. His research interests include object-oriented analysis and design, frameworks, refactoring, and components; the architecture of interactive OO software systems; aspect-oriented programming; and the employment of plug-in architectures. Lippert received an MS in computer science from the University of Hamburg. He is a member of the ACM. Contact him at lippert@acm.org.

Petra Becker-Pechau is a consultant for it-wps and a research assistant and PhD candidate in the SWT. Her research interests include OO analysis and design, ontologies, architectures and component models, plug-in architectures, framework development, and XP. Becker-Pechau received a polytechnic degree in computer science from the University of Applied Sciences in Hamburg. Contact her at becker@jwam.org.

Holger Breitling is a software architect and coach at it-wps and a research assistant and PhD candidate in the SWT. His research interests include OO analysis and design, frameworks, and components; the architecture of interactive OO software systems; the modeling of computer-supported cooperative work (CSCW) processes; and the employment of plug-in

architectures. Breitling received an MS in computer science from the University of Hamburg. Contact him at breitling@jwam.org.

Jörn Koch is a software developer and coach at it-wps. His research interests include OO analysis and design, refactoring, frameworks, and components; and architectures of CSCW systems. Koch received an MS in computer science from the University of Hamburg. Contact him at koch@jwam.org.

Andreas Kornstädt is a software architect and coach at it-wps. His research interests include OO analysis and design, frameworks and user-oriented component systems; management of software development projects; and computer applications in musicology. Kornstädt received a PhD in computer science from the University of Hamburg. Contact him at kornstaedt@jwam.org.

Stefan Roock is a software architect and coach at it-wps. His research interests include OO analysis and design, frameworks, and components; the architecture of interactive OO software systems; and management of software development projects. Roock received an MS in computer science from the University of Hamburg. Contact him at roock@jwam.org.

Axel Schmoltzky is a consultant for it-wps and a senior researcher in the SWT. His research interests include OO framework development and language design, XP, and visualization of OO design. Schmoltzky received a PhD in computer science from the University of Ulm, Germany. He is a member of the ACM and the German Computer Society (GI). Contact him at schmoltzky@jwam.org.

Henning Wolf is a software architect and consultant at it-wps. His research interests include service architectures and software project management. Wolf received an MS in computer science from the University of Hamburg. Contact him at wolf@jwam.org.

Heinz Züllighoven is CEO at it-wps, a professor in the University of Hamburg's Department of Computer Science, and head of the attached Software Technology Center. His research interests include OO analysis and design, frameworks, and components; and the architecture of interactive OO software systems. Züllighoven received a PhD in computer science from the Technical University of Berlin. He is a member of the ACM and the GI. Contact him at zuellighoven@jwam.org.